

1 Methods

1.1 Overview

INTERPBENCH is a post-hoc interpretability benchmark that quantifies the geometric organization of internal feature representations under adversarial perturbations. Unlike saliency-based or attribution-based approaches, INTERPBENCH evaluates how adversarial examples populate latent representation spaces throughout a neural network.

The central hypothesis is that models with better-organized internal representations exhibit compact and well-separated manifolds associated with different adversarial misclassification pathways, whereas poorly organized representations generate larger and more entangled regions of geometric interference.

Given a trained neural network $f(\cdot)$, a test dataset

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N, \quad (1)$$

and an adversarial attack operator $\delta(\cdot)$, INTERPBENCH evaluates the network through three sequential stages:

1. extraction of intermediate representations using forward hooks,
2. construction of a shared low-dimensional feature representation using streaming principal component analysis (PCA),
3. geometric analysis of adversarial feature distributions using convex hull estimation.

The benchmark is model-agnostic and does not require retraining, architectural modification, or access to training data.

1.2 Intermediate Feature Extraction

To analyze internal representations, forward hooks are attached to all benchmark layers. Layers containing the identifiers **layer** or **block** are automatically selected, enabling compatibility with convolutional neural networks, residual networks, vision transformers, and hybrid architectures.

For a given layer l , the extracted activation is denoted as

$$\phi_l(x) \in \mathbb{R}^{d_l}. \quad (2)$$

Because different architectures produce feature tensors with different dimensions, INTERPBENCH converts every activation tensor into a fixed-length feature vector before further analysis.

For convolutional networks,

$$\phi_l(x) \in \mathbb{R}^{C \times H \times W}, \quad (3)$$

global average pooling is applied:

$$\bar{\phi}_l(x) = \frac{1}{HW} \sum_{h=1}^H \sum_{w=1}^W \phi_l(x, h, w), \quad (4)$$

producing a C -dimensional representation.

For transformer-based architectures,

$$\phi_l(x) \in \mathbb{R}^{T \times D}, \quad (5)$$

the token dimension is aggregated:

$$\bar{\phi}_l(x) = \frac{1}{T} \sum_{t=1}^T \phi_l(x, t), \quad (6)$$

yielding a D -dimensional embedding.

Immediately after extraction, activations are detached from the computational graph and transferred to host memory. This reduces GPU memory consumption while preserving numerical equivalence.

1.3 Evaluation

INTERPBENCHevaluates feature geometry using only successful adversarial examples.

For each input image x_i , projected gradient descent (PGD) generates an adversarial example

$$x_i^{adv} = x_i + \delta_i, \quad (7)$$

subject to the perturbation constraint

$$\|\delta_i\|_\infty \leq \epsilon. \quad (8)$$

Only successful attacks satisfying

$$f(x_i^{adv}) \neq y_i \quad (9)$$

are retained, since these examples correspond to actual failures of the learned representation.

In addition to the proposed geometric metrics, conventional performance measurements are reported:

- clean accuracy,
- FGSM robustness,
- PGD robustness.

These metrics provide reference points linking representation geometry with predictive robustness.

1.4 Two-pass Streaming Representation Analysis

Direct computation of geometric statistics from intermediate activations is computationally infeasible for modern networks because they may produce millions of feature vectors.

Therefore, INTERPBENCH employs a two-pass streaming procedure separating dimensionality reduction from geometric estimation.

1.4.1 Pass I: Incremental PCA Learning

During the first pass, adversarial examples are generated for each minibatch and propagated through the network.

For each layer l , the retained successful attack representations are

$$A_l = \{\phi_l(x_i^{adv})\}. \quad (10)$$

Instead of storing the full activation matrix, INTERPBENCH updates an incremental PCA model batch-by-batch:

$$P_l \leftarrow \text{IncrementalPCA}(A_l). \quad (11)$$

Before PCA projection, features are standardized using an online standard scaler:

$$\tilde{\phi} = \frac{\phi - \mu}{\sigma}. \quad (12)$$

Only

$$k = 10 \quad (13)$$

principal components are retained.

This streaming formulation avoids constructing the complete activation matrix while approximating the principal subspace obtained by batch PCA.

1.4.2 Pass II: Geometric Accumulation

During the second pass, adversarial examples are regenerated and projected into the learned PCA space:

$$z_i = P_l \tilde{\phi}_l(x_i^{adv}) \in \mathbb{R}^{10}. \quad (14)$$

For each predicted–true class pair

$$(c_p, c_t), \quad (15)$$

where c_p denotes the predicted class and c_t denotes the ground truth class, INTERPBENCH maintains an independent geometric accumulator:

$$G_{c_p, c_t}^{(l)}. \quad (16)$$

Each projected representation is assigned according to its (prediction, ground-truth) pair, allowing the benchmark to characterize distinct adversarial transition manifolds rather than merging all failures into a single distribution.

1.5 Convex Hull Geometry

Each geometric accumulator stores a bounded set of projected points.

Whenever the number of stored samples exceeds

$$M = 200, \tag{17}$$

uniform random subsampling is applied to maintain constant memory usage.

After processing all batches, each point cloud is converted into a convex hull:

$$H = \text{ConvHull}(G). \tag{18}$$

The enclosed volume is then computed:

$$V(H). \tag{19}$$

The convex hull volume represents the spatial extent occupied by a specific adversarial transition within the learned latent manifold. Smaller volumes correspond to more compact representations, whereas larger volumes indicate dispersed and less organized geometry.

For every layer, the volumes form a class-transition matrix

$$M \in \mathbb{R}^{C \times C}, \tag{20}$$

where

$$M_{ij} \tag{21}$$

corresponds to the convex hull volume generated by samples with true label j that are classified as class i .

1.6 Interpretability Metrics

Given the geometric transition matrix M , INTERPBENCH computes two complementary metrics.

1.6.1 Interpretability Score

The InterpBench Score is defined as

$$S = \frac{1}{|M_{nz}| \cdot \text{mean}(M_{nz}) + 1}, \tag{22}$$

where $|M_{nz}|$ denotes the number of non-zero entries.

This metric simultaneously penalizes:

1. the number of active adversarial transitions,
2. their average geometric extent.

Higher values indicate better separated and more compact latent representations.

1.6.2 Interpretability Integral

The Interpretability Integral measures cumulative geometric interference:

$$I = \frac{1}{\sum_{i,j} M_{ij} + 1}. \quad (23)$$

Unlike the previous metric, the integral depends only on total geometric volume and is independent of the number of populated transitions.

For both metrics, larger values indicate improved latent organization.

1.7 Benchmark Output

For every benchmark layer, INTERPBENCH reports:

- convex hull volume matrix,
- interpretability score,
- interpretability integral,
- retained PCA variance.

Layer-level statistics are averaged to obtain network-level measures:

$$S_{\text{net}} = \frac{1}{L} \sum_{l=1}^L S_l, \quad (24)$$

and

$$I_{\text{net}} = \frac{1}{L} \sum_{l=1}^L I_l. \quad (25)$$

Together with standard accuracy and robustness measurements, these quantities form a canonical benchmark report enabling comparison across architectures, datasets, training procedures, and checkpoints.

2 Notation

2.1 Notation

We summarize the mathematical notation used throughout the INTERPBENCHbenchmark.

Table 1: Notation used in INTERPBENCH.

Symbol	Description
$f(\cdot)$	Trained neural network classifier
$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$	Evaluation dataset containing N samples
x_i	Input sample
y_i	Ground-truth class label
x_i^{adv}	Adversarial example generated from x_i
δ_i	Adversarial perturbation
ϵ	Maximum allowed ℓ_∞ perturbation magnitude
$\phi_l(x)$	Raw activation tensor extracted from layer l
$\bar{\phi}_l(x)$	Aggregated feature representation after pooling
d_l	Feature dimension of layer l
L	Number of benchmarked network layers
C	Number of output classes
A_l	Set of successful adversarial activations for layer l
$\tilde{\phi}$	Standardized feature representation
μ, σ	Online feature mean and standard deviation
P_l	Learned PCA projection matrix for layer l
k	Number of retained PCA components
z_i	Low-dimensional PCA representation
c_p	Predicted class of an adversarial sample
c_t	True class label of an adversarial sample
$G_{c_p, c_t}^{(l)}$	Geometric accumulator for a class-transition pair
M	Maximum stored points per geometric accumulator
H	Convex hull constructed from projected features
$V(H)$	Volume enclosed by convex hull H
$\mathbf{M}$	Class-transition convex hull volume matrix
$\mathbf{M}_{nz}$	Non-zero elements of the volume matrix
S	Layer-level InterpBench Score
I	Layer-level Interpretability Integral
S_{net}	Network-level Interpretability Score
I_{net}	Network-level Interpretability Integral
F	Computational cost of a single forward propagation
B	Evaluation minibatch size

2.2 Class-transition Geometry

For each layer l , INTERPBENCHrepresents adversarial failures as a collection of class-transition manifolds:

$$\mathcal{G}^{(l)} = \{G_{i,j}^{(l)} \mid i, j \in \{1, \dots, C\}\}. \quad (26)$$

Each element $G_{i,j}^{(l)}$ contains projected adversarial features whose predicted class is i and whose true class is j :

$$G_{i,j}^{(l)} = \{z_n \mid f(x_n^{adv}) = i, y_n = j\}. \quad (27)$$

The corresponding geometric matrix is defined as

$$\mathbf{M}^{(l)} = \left[V(H_{i,j}^{(l)}) \right]_{i,j=1}^C. \quad (28)$$

Thus, each matrix entry represents the spatial extent of one adversarial transition pathway inside the latent representation space.

2.3 Feature Representation Mapping

For a given layer l , the complete feature transformation pipeline is

$$x \rightarrow \phi_l(x) \rightarrow \bar{\phi}_l(x) \rightarrow \tilde{\phi}_l(x) \rightarrow z_l, \quad (29)$$

where:

- $\phi_l(x)$ is the raw intermediate activation,
- $\bar{\phi}_l(x)$ is the architecture-independent pooled feature,
- $\tilde{\phi}_l(x)$ is the standardized representation,
- $z_l \in \mathbb{R}^k$ is the final PCA embedding.

2.4 Interpretability Objective

The benchmark assumes that improved latent organization corresponds to smaller adversarial transition manifolds. Therefore, the desired representation geometry satisfies:

$$V(H_{i,j}^{(l)}) \rightarrow 0 \quad (30)$$

for successful adversarial transitions.

Consequently, larger values of

$$S_l \quad \text{and} \quad I_l \quad (31)$$

indicate increasingly compact and separable internal representations.

3 Algorithm

Algorithm 1 INTERPBENCH Evaluation Procedure

Require: Trained network f , test dataset $\mathcal{D}$, number of PCA components $k = 10$, maximum hull samples $M = 200$

Ensure: Layer-wise interpretability report and network-level benchmark statistics

- 1: Register forward hooks on all benchmark layers.
- 2: Evaluate clean accuracy, FGSM robustness, and PGD robustness.

3: **Pass I: Streaming PCA Learning**

- 4: **for** each minibatch $(x_i, y_i) \in \mathcal{D}$ **do**
- 5: Generate PGD adversarial examples:

$$x_i^{adv} = x_i + \delta_i$$

- 6: Forward propagate x_i^{adv} through f .
- 7: Extract intermediate activations $\phi_l(x_i^{adv})$ for every benchmark layer l .
- 8: Retain only successful adversarial samples:

$$f(x_i^{adv}) \neq y_i$$

- 9: Update online feature standardizers.
- 10: Update Incremental PCA models:

$$P_l \leftarrow \text{IncrementalPCA}(A_l)$$

- 11: **end for**

12: **Pass II: Geometric Accumulation**

- 13: **for** each minibatch $(x_i, y_i) \in \mathcal{D}$ **do**
- 14: Regenerate PGD adversarial examples.
- 15: Forward propagate adversarial samples.
- 16: Extract intermediate representations.
- 17: Standardize features using learned statistics.
- 18: Project representations into PCA space:

$$z_i = P_l \tilde{\phi}_l(x_i^{adv})$$

- 19: Determine predicted class:

$$c_p = f(x_i^{adv})$$

- 20: Assign projected feature according to (c_p, y_i) transition pair.
- 21: Update geometric accumulator:

$$G_{c_p, y_i}^{(l)}$$

- 22: If $|G_{c_p, y_i}^{(l)}| > M$, perform uniform random subsampling.
- 23: **end for**

24: **for** each benchmark layer l **do**

- 25: Construct convex hulls:

$$H_{ij}^{(l)} = \text{ConvHull}(G_{i,j}^{(l)})$$

- 26: Compute class-transition volume matrix:

$$M_{ij}^{(l)} = V(H_{ij}^{(l)})$$

- 27: Calculate Interpretability Score:

$$S_l = \frac{1}{|M_{nz}^{(l)}| \text{mean}(M_{nz}^{(l)}) + 1}$$

4 Computational Complexity

We analyze the computational requirements of INTERPBENCH with respect to the number of evaluation samples, benchmark layers, feature dimensionality, number of classes, and geometric reservoir size.

Let:

- N denote the number of evaluation samples,
- L denote the number of benchmarked layers,
- d denote the feature dimension before dimensionality reduction,
- k denote the retained PCA dimension ($k = 10$),
- C denote the number of classes,
- M denote the maximum number of stored points per class pair ($M = 200$),
- B denote the evaluation minibatch size.

4.1 Time Complexity

The feature extraction stage requires two complete forward passes through the network: one pass for learning the PCA representation and one pass for geometric accumulation.

The forward propagation cost is therefore

$$O(2NF), \quad (32)$$

where F represents the computational cost of a single forward pass.

During the first pass, online feature standardization and incremental PCA updates process each extracted feature vector once. For all benchmark layers, this contributes

$$O(NLdk). \quad (33)$$

Since $k \ll d$, dimensionality reduction introduces only a small overhead relative to feature extraction.

During the second pass, projected representations are computed using the learned PCA bases:

$$z_i = P_l \tilde{\phi}_l(x_i^{adv}), \quad (34)$$

which introduces an additional complexity of

$$O(NLdk). \quad (35)$$

Each projected feature is assigned to one of the predicted–true class transition accumulators. The corresponding complexity is

$$O(NLC^2). \quad (36)$$

Although the implementation maintains C^2 transition categories, C is fixed for a given dataset. Therefore, this component scales linearly with the number of evaluation samples.

After all samples have been processed, convex hulls are constructed for each class-transition accumulator. Since each accumulator contains at most M points, the total geometric computation is

$$O(LC^2M \log M). \quad (37)$$

Because M is fixed, this term remains effectively constant.

Combining all components, the dominant time complexity of INTERPBENCH is

$$\boxed{O(NF + NLdk)} \quad (38)$$

demonstrating linear scaling with the number of evaluation samples.

4.2 Space Complexity

INTERPBENCH is designed around bounded-memory streaming computation.

The main memory requirements are:

1. Incremental PCA statistics:

$$O(Ldk) \quad (39)$$

2. Convex hull reservoirs:

$$O(LC^2M) \quad (40)$$

3. One minibatch of intermediate activations:

$$O(Bd) \quad (41)$$

Unlike approaches that store all intermediate activations, which require

$$O(Nd) \quad (42)$$

memory, INTERPBENCH never materializes the complete activation matrix.

Activations are detached immediately after extraction, transferred to host memory, and processed incrementally.

Therefore, the peak memory complexity is

$$\boxed{O(Ldk + LC^2M + Bd)} \quad (43)$$

which is independent of the total evaluation set size N .

This bounded-memory design enables evaluation of large datasets and deep architectures while maintaining predictable computational requirements.

5 Hyperparameters

5.1 Benchmark Hyperparameters

All experiments using INTERPBENCH employ the fixed hyperparameter configuration shown in Table 2. These parameters define the adversarial generation procedure, feature processing pipeline, dimensional reduction strategy, and geometric estimation procedure.

Table 2: Default hyperparameter configuration used by INTERPBENCH.

Parameter	Value	Description
Adversarial attack	PGD	Adversarial sample generation method
PGD perturbation budget (ϵ)	8/255	Maximum ℓ_∞ perturbation magnitude
PGD step size (α)	2/255	Gradient update magnitude
PGD iterations	10	Number of attack optimization steps
PCA components (k)	10	Number of retained principal components
Feature normalization	StandardScaler	Online feature standardization
PCA algorithm	Incremental PCA	Streaming dimensionality reduction method
Geometry estimator	Convex Hull	Latent geometry representation
Maximum hull points (M)	200	Memory-bounded point reservoir size
CNN feature aggregation	Global Average Pooling	Spatial feature reduction
Transformer feature aggregation	Token Averaging	Sequence dimension reduction
Evaluation strategy	Two-pass streaming	Memory-efficient feature processing

5.2 Configuration

Adversarial examples are generated using projected gradient descent:

$$x^{t+1} = \Pi_{x+\mathcal{B}_\epsilon} (x^t + \alpha \text{sign}(\nabla_x \mathcal{L}(f(x^t)), y)) , \quad (44)$$

where $\epsilon = 8/255$ defines the maximum perturbation radius, $\alpha = 2/255$ defines the update step size, and the optimization is performed for ten iterations.

Only adversarial examples that successfully change the predicted class are retained for geometric analysis:

$$f(x^{adv}) \neq y. \quad (45)$$

5.3 Feature Processing Configuration

Intermediate activations are converted into architecture-independent representations before geometric analysis.

For convolutional architectures, spatial dimensions are removed using global average pooling:

$$\bar{\phi}_l = \frac{1}{HW} \sum_{h,w} \phi_l(h, w). \quad (46)$$

For transformer architectures, token-level representations are averaged:

$$\bar{\phi}_l = \frac{1}{T} \sum_t \phi_l(t). \quad (47)$$

5.4 Streaming Constraints

To maintain bounded memory usage, INTERPBENCH applies two constraints:

1. PCA fitting is performed incrementally using minibatch updates.
2. Convex hull accumulators retain at most $M = 200$ samples per class-transition pair.

These constraints ensure that memory requirements remain independent of the total number of evaluation samples.